

# Linear Models for Classification in Machine Learning

## 1. Introduction to Classification

Classification is a supervised learning task where the goal is to predict discrete class labels for input data. Unlike regression, which predicts continuous values, classification assigns inputs to predefined categories or classes.

### Key Concepts:

- **Binary Classification:** Two classes (e.g., spam/not spam, disease/no disease)
- **Multi-class Classification:** More than two classes (e.g., digit recognition 0-9)
- **Features:** Input variables  $\mathbf{x} = (x_1, x_2, \dots, x_D)^T$
- **Target:** Class label  $y \in \{C_1, C_2, \dots, C_K\}$

Linear models for classification construct decision boundaries that are linear functions of the input features. Despite their simplicity, these models are powerful, interpretable, and form the foundation for understanding more complex classification methods[1].

## 2. Decision Boundaries and Linear Discriminants

### 2.1 Linear Decision Boundary

A linear decision boundary in  $D$ -dimensional input space is defined by:

$$y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0 = \sum_{i=1}^D w_i x_i + w_0 = 0$$

where:

- $\mathbf{w} = (w_1, w_2, \dots, w_D)^T$  is the weight vector (normal to the decision surface)
- $w_0$  is the bias term (also called threshold or intercept)
- $\mathbf{x} = (x_1, x_2, \dots, x_D)^T$  is the input feature vector
- $D$  is the dimensionality of the input space

### Detailed Explanation:

The function  $y(\mathbf{x})$  is a linear combination of the input features. The decision boundary is the set of points where  $y(\mathbf{x}) = 0$ , which forms a hyperplane in  $D$ -dimensional space:

- In 2D: the boundary is a line
- In 3D: the boundary is a plane
- In higher dimensions: the boundary is a hyperplane

**Classification Rule:**

$$\text{Class} = \begin{cases} C_1 & \text{if } y(\mathbf{x}) \geq 0 \\ C_2 & \text{if } y(\mathbf{x}) < 0 \end{cases}$$

The sign of  $y(\mathbf{x})$  determines which side of the hyperplane the point lies on. Points exactly on the boundary have  $y(\mathbf{x}) = 0$ .

**Geometric Properties:**

1. **Normal Vector:** The weight vector  $\mathbf{w}$  is perpendicular (normal) to the decision boundary.
2. **Bias Term Effect:** The bias  $w_0$  shifts the boundary parallel to itself without changing its orientation. Specifically:
  - If  $w_0 = 0$ , the boundary passes through the origin
  - If  $w_0 > 0$ , the boundary shifts in the direction of  $-\mathbf{w}$
  - If  $w_0 < 0$ , the boundary shifts in the direction of  $+\mathbf{w}$
3. **Homogeneous Coordinates:** We can incorporate the bias by augmenting the weight vector and input:  $\tilde{\mathbf{w}} = (w_0, w_1, \dots, w_D)^T$  and  $\tilde{\mathbf{x}} = (1, x_1, \dots, x_D)^T$ , giving  $y(\mathbf{x}) = \tilde{\mathbf{w}}^T \tilde{\mathbf{x}}$ .

## 2.2 Geometric Interpretation

The perpendicular distance from a point  $\mathbf{x}$  to the decision boundary is:

$$r = \frac{y(\mathbf{x})}{\|\mathbf{w}\|} = \frac{|\mathbf{w}^T \mathbf{x} + w_0|}{\sqrt{w_1^2 + w_2^2 + \dots + w_D^2}}$$

**Derivation:**

Consider a point  $\mathbf{x}$  and its projection  $\mathbf{x}_\perp$  onto the decision boundary. The vector from  $\mathbf{x}_\perp$  to  $\mathbf{x}$  is parallel to  $\mathbf{w}$ :

$$\mathbf{x} = \mathbf{x}_\perp + r \frac{\mathbf{w}}{\|\mathbf{w}\|}$$

Since  $\mathbf{x}_\perp$  is on the boundary,  $\mathbf{w}^T \mathbf{x}_\perp + w_0 = 0$ . Substituting:

$$\begin{aligned} y(\mathbf{x}) &= \mathbf{w}^T \mathbf{x} + w_0 = \mathbf{w}^T \left( \mathbf{x}_\perp + r \frac{\mathbf{w}}{\|\mathbf{w}\|} \right) + w_0 \\ &= \mathbf{w}^T \mathbf{x}_\perp + w_0 + r \frac{\mathbf{w}^T \mathbf{w}}{\|\mathbf{w}\|} = 0 + r \frac{\|\mathbf{w}\|^2}{\|\mathbf{w}\|} = r \|\mathbf{w}\| \end{aligned}$$

Therefore:

$$r = \frac{y(\mathbf{x})}{\|\mathbf{w}\|}$$

**Interpretation:**

- If  $y(\mathbf{x}) > 0$ : point is on the positive side (direction of  $\mathbf{w}$ ), distance is  $r = \frac{y(\mathbf{x})}{\|\mathbf{w}\|}$
- If  $y(\mathbf{x}) < 0$ : point is on the negative side, distance is  $r = \frac{-y(\mathbf{x})}{\|\mathbf{w}\|}$
- If  $y(\mathbf{x}) = 0$ : point is on the boundary, distance is 0

The signed distance (with sign indicating which side) is  $\frac{y(\mathbf{x})}{\|\mathbf{w}\|}$ .

**Example Problem 2.1:** Find the distance from point  $\mathbf{x} = (3,4)^T$  to the decision boundary defined by  $2x_1 + x_2 - 5 = 0$ .

**Solution:**

Given:  $\mathbf{x} = (3,4)^T$ , decision boundary:  $2x_1 + x_2 - 5 = 0$

Weight vector:  $\mathbf{w} = (2,1)^T$ , bias:  $w_0 = -5$

**Step 1:** Compute  $y(\mathbf{x})$ :

$$y(\mathbf{x}) = 2(3) + 1(4) - 5 = 6 + 4 - 5 = 5$$

**Step 2:** Compute  $\|\mathbf{w}\|$ :

$$\|\mathbf{w}\| = \sqrt{2^2 + 1^2} = \sqrt{4 + 1} = \sqrt{5} \approx 2.236$$

**Step 3:** Compute the signed distance:

$$r = \frac{y(\mathbf{x})}{\|\mathbf{w}\|} = \frac{5}{\sqrt{5}} = \frac{5\sqrt{5}}{5} = \sqrt{5} \approx 2.236$$

**Conclusion:** The point  $\mathbf{x} = (3,4)^T$  is at distance  $\sqrt{5} \approx 2.236$  units from the boundary, on the positive side (Class  $C_1$ ). Since  $y(\mathbf{x}) = 5 > 0$ , the point would be classified as  $C_1$ .

**Example Problem 2.2:** Consider the decision boundary  $x_1 - 2x_2 + 3 = 0$ . Find the perpendicular distance from the following points and classify them:

(a)  $\mathbf{x}_a = (1,1)^T$

(b)  $\mathbf{x}_b = (-1,2)^T$

(c)  $\mathbf{x}_c = (3,3)^T$

**Solution:**

Given:  $\mathbf{w} = (1, -2)^T$ ,  $w_0 = 3$ ,  $\|\mathbf{w}\| = \sqrt{1^2 + (-2)^2} = \sqrt{5}$

**(a) For  $\mathbf{x}_a = (1,1)^T$ :**

$$y(\mathbf{x}_a) = 1(1) - 2(1) + 3 = 1 - 2 + 3 = 2$$

$$r_a = \frac{2}{\sqrt{5}} = \frac{2\sqrt{5}}{5} \approx 0.894$$

Classification:  $C_1$  (since  $y(\mathbf{x}_a) > 0$ ), distance  $\approx 0.894$  units.

**(b) For  $\mathbf{x}_b = (-1, 2)^T$ :**

$$y(\mathbf{x}_b) = 1(-1) - 2(2) + 3 = -1 - 4 + 3 = -2$$

$$r_b = \frac{-2}{\sqrt{5}} = \frac{-2\sqrt{5}}{5} \approx -0.894$$

Classification:  $C_2$  (since  $y(\mathbf{x}_b) < 0$ ), distance  $\approx 0.894$  units (on negative side).

**(c) For  $\mathbf{x}_c = (3, 3)^T$ :**

$$y(\mathbf{x}_c) = 1(3) - 2(3) + 3 = 3 - 6 + 3 = 0$$

$$r_c = \frac{0}{\sqrt{5}} = 0$$

Classification: On the boundary (exactly on the decision surface), distance = 0.

**Example Problem 2.3:** Two parallel decision boundaries are defined by  $2x_1 + x_2 - 4 = 0$  and  $2x_1 + x_2 + 2 = 0$ . Find the perpendicular distance between these two boundaries.

**Solution:**

Both boundaries have the same weight vector  $\mathbf{w} = (2, 1)^T$ , so they are parallel.

**Method 1** (Using any point on one boundary):

Take a point on the first boundary, say  $\mathbf{x}_0 = (2, 0)^T$  (verify:  $2(2) + 0 - 4 = 0$  ✓).

Find its distance to the second boundary  $2x_1 + x_2 + 2 = 0$ :

$$y(\mathbf{x}_0) = 2(2) + 1(0) + 2 = 4 + 0 + 2 = 6$$

$$\|\mathbf{w}\| = \sqrt{2^2 + 1^2} = \sqrt{5}$$

$$d = \frac{|y(\mathbf{x}_0)|}{\|\mathbf{w}\|} = \frac{6}{\sqrt{5}} = \frac{6\sqrt{5}}{5} \approx 2.683$$

**Method 2** (Using bias difference):

For parallel hyperplanes with the same  $\mathbf{w}$  but different biases  $w_{0,1}$  and  $w_{0,2}$ :

$$d = \frac{|w_{0,1} - w_{0,2}|}{\|\mathbf{w}\|}$$

Here:  $w_{0,1} = -4$ ,  $w_{0,2} = 2$

$$d = \frac{|-4 - 2|}{\sqrt{5}} = \frac{6}{\sqrt{5}} = \frac{6\sqrt{5}}{5} \approx 2.683$$

**Conclusion:** The perpendicular distance between the two parallel boundaries is  $\frac{6\sqrt{5}}{5} \approx 2.683$  units.

**Example Problem 2.4:** A decision boundary is defined by  $3x_1 - 4x_2 + 10 = 0$ . Find:

- (a) A unit normal vector to the boundary
- (b) The distance from the origin to the boundary
- (c) A point on the boundary closest to the origin

**Solution:**

Given:  $\mathbf{w} = (3, -4)^T$ ,  $w_0 = 10$

**(a) Unit normal vector:**

$$\|\mathbf{w}\| = \sqrt{3^2 + (-4)^2} = \sqrt{9 + 16} = 5$$

$$\hat{\mathbf{w}} = \frac{\mathbf{w}}{\|\mathbf{w}\|} = \frac{1}{5}(3, -4)^T = (0.6, -0.8)^T$$

**(b) Distance from origin  $\mathbf{x}_0 = (0,0)^T$ :**

$$y(\mathbf{x}_0) = 3(0) - 4(0) + 10 = 10$$

$$r = \frac{|y(\mathbf{x}_0)|}{\|\mathbf{w}\|} = \frac{10}{5} = 2$$

Distance from origin to boundary: 2 units.

**(c) Closest point on boundary:**

The closest point is along the normal direction from the origin:

$$\mathbf{x}_{\text{closest}} = \mathbf{x}_0 - r\hat{\mathbf{w}} = (0,0)^T - 2(0.6, -0.8)^T = (-1.2, 1.6)^T$$

**Verification:** Check if this point is on the boundary:

$$3(-1.2) - 4(1.6) + 10 = -3.6 - 6.4 + 10 = 0 \quad \checkmark$$

The closest point on the boundary to the origin is  $(-1.2, 1.6)^T$ .

**Example Problem 2.5:** Given three points  $\mathbf{x}_1 = (1,2)^T$ ,  $\mathbf{x}_2 = (2,3)^T$ ,  $\mathbf{x}_3 = (4,1)^T$ , and a decision boundary  $x_1 + x_2 - 4 = 0$ . Which point is farthest from the boundary?

**Solution:**

Given:  $\mathbf{w} = (1,1)^T$ ,  $w_0 = -4$ ,  $\|\mathbf{w}\| = \sqrt{1^2 + 1^2} = \sqrt{2}$

**For  $\mathbf{x}_1 = (1,2)^T$ :**

$$y(\mathbf{x}_1) = 1 + 2 - 4 = -1$$

$$r_1 = \frac{|-1|}{\sqrt{2}} = \frac{1}{\sqrt{2}} = \frac{\sqrt{2}}{2} \approx 0.707$$

For  $\mathbf{x}_2 = (2,3)^T$ :

$$y(\mathbf{x}_2) = 2 + 3 - 4 = 1$$

$$r_2 = \frac{|1|}{\sqrt{2}} = \frac{1}{\sqrt{2}} = \frac{\sqrt{2}}{2} \approx 0.707$$

For  $\mathbf{x}_3 = (4,1)^T$ :

$$y(\mathbf{x}_3) = 4 + 1 - 4 = 1$$

$$r_3 = \frac{|1|}{\sqrt{2}} = \frac{1}{\sqrt{2}} = \frac{\sqrt{2}}{2} \approx 0.707$$

**Conclusion:** Interestingly, all three points are equidistant from the boundary at  $\frac{\sqrt{2}}{2} \approx 0.707$  units. However,  $\mathbf{x}_1$  is on the negative side (Class  $C_2$ ), while  $\mathbf{x}_2$  and  $\mathbf{x}_3$  are on the positive side (Class  $C_1$ ).

## 3. Fisher's Linear Discriminant Analysis (LDA)

Advantages and Disadvantages of FLD

### 3.1 Motivation

Fisher's Linear Discriminant Analysis (LDA) finds a linear projection that maximizes class separability. The key idea is to project high-dimensional data onto a one-dimensional space such that the projected classes are well-separated[2].

Fisher's Linear Discriminant Analysis (LDA)

Linear Discriminant Analysis

### 3.2 Fisher's Criterion

Advantages

For a two-class problem, Fisher's criterion seeks to maximize the ratio:

$$J(\mathbf{w}) = \frac{(\tilde{m}_1 - \tilde{m}_2)^2}{\tilde{s}_1^2 + \tilde{s}_2^2}$$

1. Reduces dimensionality of data
2. Improves class separation
3. Fast and computationally efficient
4. Works well with small datasets
5. Reduces overfitting
6. Useful for classification problems
7. Easy to interpret

where:

- $\tilde{m}_k = \mathbf{w}^T \mathbf{m}_k$  is the projected mean of class  $k$
- $\tilde{s}_k^2$  is the projected within-class variance of class  $k$
- $\mathbf{m}_k = \frac{1}{N_k} \sum_{\mathbf{x} \in C_k} \mathbf{x}$  is the mean of class  $k$

Disadvantages

1. Assumes data is normally distributed
2. Assumes equal covariance among classes
3. Performs poorly on non-linear data
4. Sensitive to outliers
5. Cannot capture complex relationships
6. Number of components is limited to number of classes - 1
7. Not suitable when classes overlap heavily

### 3.3 Within-Class and Between-Class Scatter

Within-class scatter matrix:

$$\mathbf{S}_W = \sum_{k=1}^2 \sum_{\mathbf{x} \in C_k} (\mathbf{x} - \mathbf{m}_k)(\mathbf{x} - \mathbf{m}_k)^T$$

Difference between within class and Between class

### Between-class scatter matrix:

$$\mathbf{S}_B = (\mathbf{m}_2 - \mathbf{m}_1)(\mathbf{m}_2 - \mathbf{m}_1)^T$$

Fisher's criterion can be rewritten as:

$$J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$$

$\mathbf{w}$  = Projection Vector

## 3.4 Optimal Solution

The weight vector that maximizes  $J(\mathbf{w})$  is:

$$\mathbf{w} \propto \mathbf{S}_W^{-1}(\mathbf{m}_2 - \mathbf{m}_1)$$

This is the **Fisher's linear discriminant direction**.

**Example Problem 3.1:** Given two classes with the following properties:

Class 1:  $\mathbf{m}_1 = (1,2)^T$ ,  $N_1 = 3$

Class 2:  $\mathbf{m}_2 = (4,6)^T$ ,  $N_2 = 3$

Within-class scatter:  $\mathbf{S}_W = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$

Find Fisher's linear discriminant direction.

**Solution:**

$$\mathbf{m}_2 - \mathbf{m}_1 = \begin{bmatrix} 4 \\ 6 \end{bmatrix} - \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

$$\mathbf{S}_W^{-1} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$$

$$\mathbf{w} \propto \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 1.5 \\ 2.0 \end{bmatrix}$$

Within-Class Scatter Matrix

- \* Measures spread within the same class
- \* Shows class compactness
- \* Smaller value is better
- \* Denoted by ( $\mathbf{S}_W$ )

Formula:

Between-Class Scatter Matrix

- \* Measures separation between different classes
- \* Shows class distinguishability
- \* Larger value is better
- \* Denoted by ( $\mathbf{S}_B$ )

Formula:

The Fisher's discriminant direction is  $\mathbf{w} = (1.5, 2.0)^T$  (or any scalar multiple).

## 4. Probabilistic Generative Models

### 4.1 Bayes' Theorem for Classification

Probabilistic generative models model the class-conditional densities  $p(\mathbf{x}|C_k)$  and class priors  $p(C_k)$ , then use Bayes' theorem:

$$p(C_k|\mathbf{x}) = \frac{p(\mathbf{x}|C_k)p(C_k)}{p(\mathbf{x})}$$

where  $p(\mathbf{x}) = \sum_j p(\mathbf{x}|C_j)p(C_j)$  is the marginal likelihood.

### 4.2 Gaussian Class-Conditional Densities

Assume each class has a Gaussian distribution:

$$p(\mathbf{x}|C_k) = \frac{1}{(2\pi)^{D/2} |\boldsymbol{\Sigma}_k|^{1/2}} \exp \left( -\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) \right)$$

where  $\boldsymbol{\mu}_k$  is the mean and  $\boldsymbol{\Sigma}_k$  is the covariance matrix of class  $k$ .

### 4.3 Linear Discriminant Analysis (Shared Covariance)

When all classes share the same covariance matrix  $\boldsymbol{\Sigma}_k = \boldsymbol{\Sigma}$ , the decision boundary between classes  $C_i$  and  $C_j$  is linear:

$$\ln \frac{p(C_i|\mathbf{x})}{p(C_j|\mathbf{x})} = \mathbf{w}_{ij}^T \mathbf{x} + w_{ij0}$$

where:

$$\mathbf{w}_{ij} = \boldsymbol{\Sigma}^{-1} (\boldsymbol{\mu}_i - \boldsymbol{\mu}_j)$$

$$w_{ij0} = -\frac{1}{2} \boldsymbol{\mu}_i^T \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_i + \frac{1}{2} \boldsymbol{\mu}_j^T \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_j + \ln \frac{p(C_i)}{p(C_j)}$$

This is called **Linear Discriminant Analysis (LDA)** in the generative model context.

### 4.4 Quadratic Discriminant Analysis (Different Covariances)

When classes have different covariance matrices  $\boldsymbol{\Sigma}_k$ , the decision boundaries become quadratic functions of  $\mathbf{x}$ , leading to **Quadratic Discriminant Analysis (QDA)**.

**Example Problem 4.1:** Two classes with equal priors  $p(C_1) = p(C_2) = 0.5$  have Gaussian distributions:

$$\text{Class 1: } \boldsymbol{\mu}_1 = (0,0)^T, \boldsymbol{\Sigma} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\text{Class 2: } \boldsymbol{\mu}_2 = (3,0)^T, \boldsymbol{\Sigma} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Find the decision boundary.

**Solution:**

Since covariances are equal, we use LDA:

$$\mathbf{w} = \boldsymbol{\Sigma}^{-1} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} -3 \\ 0 \end{bmatrix} = \begin{bmatrix} -3 \\ 0 \end{bmatrix}$$

$$w_0 = -\frac{1}{2} \boldsymbol{\mu}_1^T \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_1 + \frac{1}{2} \boldsymbol{\mu}_2^T \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_2 + \ln(1)$$



$$w_0 = 0 + \frac{1}{2}(9) + 0 = 4.5$$

Decision boundary:  $-3x_1 + 4.5 = 0$  or  $x_1 = 1.5$

Points with  $x_1 < 1.5$  belong to Class 1, and  $x_1 \geq 1.5$  belong to Class 2.

## 5. Probabilistic Discriminative Models

### 5.1 Logistic Regression

Logistic regression directly models the posterior probability  $p(C_1|\mathbf{x})$  using the logistic sigmoid function:

$$p(C_1|\mathbf{x}) = \sigma(y(\mathbf{x})) = \frac{1}{1 + \exp(-y(\mathbf{x}))}$$

where  $y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0$  is the linear discriminant function, and:

$$\sigma(a) = \frac{1}{1 + \exp(-a)}$$

is the **logistic sigmoid function**.

### 5.2 Properties of the Sigmoid Function

**Derivatives:**

$$\frac{d\sigma}{da} = \sigma(a)(1 - \sigma(a))$$

**Symmetry:**

$$\sigma(-a) = 1 - \sigma(a)$$

**Inverse (logit):**

$$a = \ln \frac{\sigma}{1 - \sigma}$$

## 5.3 Maximum Likelihood Estimation for Logistic Regression

Given a dataset  $\{(\mathbf{x}_n, t_n)\}_{n=1}^N$  where  $t_n \in \{0,1\}$ , the likelihood is:

$$p(\mathbf{t}|\mathbf{w}) = \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n}$$

where  $y_n = p(C_1|\mathbf{x}_n) = \sigma(\mathbf{w}^T \mathbf{x}_n)$ .

The negative log-likelihood (cross-entropy error) is:

$$E(\mathbf{w}) = -\ln p(\mathbf{t}|\mathbf{w}) = -\sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln (1 - y_n)\}$$

## 5.4 Gradient Descent for Logistic Regression

The gradient of the error function is:

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N (y_n - t_n) \mathbf{x}_n$$

**Gradient descent update rule:**

$$\mathbf{w}^{(\text{new})} = \mathbf{w}^{(\text{old})} - \eta \nabla E(\mathbf{w})$$

where  $\eta$  is the learning rate.

**Example Problem 5.1:** Given a training dataset with 2 features:

| $x_1$ | $x_2$ | $t$ (label) |
|-------|-------|-------------|
| 1     | 2     | 1           |
| 2     | 1     | 1           |
| 2     | 3     | 0           |
| 3     | 2     | 0           |

Initialize weights  $\mathbf{w} = (0,0,0)^T$  (including bias). Perform one iteration of gradient descent with learning rate  $\eta = 0.1$ .

**Solution:**

Add bias term  $x_0 = 1$  to each input:

$$\mathbf{x}_1 = (1,1,2)^T, \mathbf{x}_2 = (1,2,1)^T, \mathbf{x}_3 = (1,2,3)^T, \mathbf{x}_4 = (1,3,2)^T$$

For initial  $\mathbf{w} = (0,0,0)^T$ :

$$y_n = \sigma(\mathbf{w}^T \mathbf{x}_n) = \sigma(0) = 0.5 \text{ for all } n$$

Gradient:

$$\begin{aligned} \nabla E(\mathbf{w}) &= (0.5 - 1)\mathbf{x}_1 + (0.5 - 1)\mathbf{x}_2 + (0.5 - 0)\mathbf{x}_3 + (0.5 - 0)\mathbf{x}_4 \\ &= -0.5(1,1,2)^T - 0.5(1,2,1)^T + 0.5(1,2,3)^T + 0.5(1,3,2)^T \\ &= (-0.5 - 0.5 + 0.5 + 0.5, -0.5 - 1 + 1 + 1.5, -1 - 0.5 + 1.5 + 1)^T = (0,1,1)^T \end{aligned}$$

Update:

$$\mathbf{w}^{(\text{new})} = (0,0,0)^T - 0.1(0,1,1)^T = (0, -0.1, -0.1)^T$$

After one iteration:  $\mathbf{w} = (0, -0.1, -0.1)^T$

## 5.5 Multi-class Logistic Regression (Softmax)

For  $K > 2$  classes, we use the **softmax function**:

$$p(C_k|\mathbf{x}) = \frac{\exp(y_k(\mathbf{x}))}{\sum_{j=1}^K \exp(y_j(\mathbf{x}))}$$

where  $y_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x} + w_{k0}$  for class  $k$ .

The cross-entropy error for multi-class classification is:

$$E(\mathbf{w}_1, \dots, \mathbf{w}_K) = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln p(C_k|\mathbf{x}_n)$$

where  $t_{nk}$  is 1 if  $\mathbf{x}_n$  belongs to class  $k$ , and 0 otherwise.

## 6. Perceptron Algorithm

### 6.1 Perceptron Model

The perceptron is a simple binary linear classifier:

$$y(\mathbf{x}) = f(\mathbf{w}^T \mathbf{x})$$

where  $f(a)$  is a step function:

$$f(a) = \begin{cases} +1 & \text{if } a \geq 0 \\ -1 & \text{if } a < 0 \end{cases}$$

### 6.2 Perceptron Learning Algorithm

The perceptron uses the following update rule when a misclassification occurs:

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} + \eta t_n \mathbf{x}_n$$

where:

- $\eta > 0$  is the learning rate
- $t_n \in \{-1, +1\}$  is the true label
- $\mathbf{x}_n$  is the misclassified input

**Perceptron Convergence Theorem:** If the training data is linearly separable, the perceptron algorithm converges in a finite number of steps.

**Example Problem 6.1:** Train a perceptron on the following linearly separable dataset:

| $x_1$ | $x_2$ | $t$ (label) |
|-------|-------|-------------|
| 0     | 0     | -1          |

|   |   |    |
|---|---|----|
| 0 | 1 | +1 |
| 1 | 0 | +1 |
| 1 | 1 | +1 |

Initialize  $\mathbf{w} = (0,0,0)^T$  (including bias), use  $\eta = 1$ .

### Solution:

Add bias:  $\mathbf{x}_1 = (1,0,0)^T$ ,  $\mathbf{x}_2 = (1,0,1)^T$ ,  $\mathbf{x}_3 = (1,1,0)^T$ ,  $\mathbf{x}_4 = (1,1,1)^T$

### Iteration 1:

- $\mathbf{x}_1: \mathbf{w}^T \mathbf{x}_1 = 0$ , predict +1, correct (no update)
- $\mathbf{x}_2: \mathbf{w}^T \mathbf{x}_2 = 0$ , predict +1, correct (no update)
- $\mathbf{x}_3: \mathbf{w}^T \mathbf{x}_3 = 0$ , predict +1, correct (no update)
- $\mathbf{x}_4: \mathbf{w}^T \mathbf{x}_4 = 0$ , predict +1, correct (no update)

Wait, with  $\mathbf{w} = (0,0,0)^T$ , all predictions are on the boundary. Let's be more careful.

### Corrected Iteration 1:

- $\mathbf{x}_1: \mathbf{w}^T \mathbf{x}_1 = 0 \geq 0$ , predict +1, true label -1, **misclassified**
  - Update:  $\mathbf{w} = (0,0,0)^T + 1 \cdot (-1) \cdot (1,0,0)^T = (-1,0,0)^T$
- $\mathbf{x}_2: \mathbf{w}^T \mathbf{x}_2 = -1 < 0$ , predict -1, true label +1, **misclassified**
  - Update:  $\mathbf{w} = (-1,0,0)^T + 1 \cdot (+1) \cdot (1,0,1)^T = (0,0,1)^T$
- $\mathbf{x}_3: \mathbf{w}^T \mathbf{x}_3 = 0 \geq 0$ , predict +1, true label +1, correct
- $\mathbf{x}_4: \mathbf{w}^T \mathbf{x}_4 = 1 \geq 0$ , predict +1, true label +1, correct

### Iteration 2:

- $\mathbf{x}_1: \mathbf{w}^T \mathbf{x}_1 = 0 \geq 0$ , predict +1, true label -1, **misclassified**
  - Update:  $\mathbf{w} = (0,0,1)^T + 1 \cdot (-1) \cdot (1,0,0)^T = (-1,0,1)^T$
- Continue this process...

The perceptron will converge to a separating hyperplane after a finite number of iterations.

## 7. Comparison of Linear Classification Methods

| Method       | Type       | Decision Boundary | Key Properties             |
|--------------|------------|-------------------|----------------------------|
| Fisher's LDA | Generative | Linear            | Maximizes class separation |

|                     |                |           |                                         |
|---------------------|----------------|-----------|-----------------------------------------|
| Gaussian LDA        | Generative     | Linear    | Assumes Gaussian, shared covariance     |
| Gaussian QDA        | Generative     | Quadratic | Assumes Gaussian, different covariances |
| Logistic Regression | Discriminative | Linear    | Models posterior directly               |
| Perceptron          | Discriminative | Linear    | Simple, only for separable data         |

Table 1: Comparison of linear classification methods

## 8. Advantages and Limitations

### 8.1 Advantages

- **Interpretability:** Weights can be interpreted as feature importance
- **Computational efficiency:** Fast training and prediction
- **Low sample complexity:** Work well with limited data
- **Probabilistic outputs:** Logistic regression provides calibrated probabilities

### 8.2 Limitations

- **Linear decision boundaries:** Cannot handle complex non-linear patterns
- **Feature engineering required:** May need manual feature transformations
- **Sensitive to outliers:** Especially in generative models
- **Class imbalance:** May require careful handling of imbalanced datasets

## 9. Practical Considerations

### 9.1 Feature Scaling

Linear models are sensitive to feature scales. **Standardization** is recommended:

$$x_j^{\text{scaled}} = \frac{x_j - \mu_j}{\sigma_j}$$

where  $\mu_j$  is the mean and  $\sigma_j$  is the standard deviation of feature  $j$ .

### 9.2 Regularization

To prevent overfitting, add a regularization term to the loss function:

**L2 regularization (Ridge):**

$$E(\mathbf{w}) = E_0(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

**L1 regularization (Lasso):**

$$E(\mathbf{w}) = E_0(\mathbf{w}) + \lambda \|\mathbf{w}\|_1$$

where  $\lambda > 0$  is the regularization parameter.

## 9.3 Model Selection

Use cross-validation to select:

- Regularization parameter  $\lambda$
- Feature subsets
- Preprocessing techniques

## 10. Conclusion

Linear models for classification provide a solid foundation for understanding machine learning classification tasks. While they have limitations in handling complex non-linear patterns, they remain valuable due to their interpretability, efficiency, and effectiveness on many real-world problems. Understanding these models prepares students for more advanced techniques such as kernel methods, neural networks, and ensemble methods covered later in the course.

**Key Takeaways:**

- Fisher's LDA finds optimal projection for class separation
- Generative models (LDA/QDA) model class-conditional densities
- Discriminative models (logistic regression) model posteriors directly
- Perceptron is simple but limited to linearly separable data
- Regularization and proper feature scaling are essential for practical applications

## References

- [1] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- [2] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.
- [3] Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press.
- [4] Duda, R. O., Hart, P. E., & Stork, D. G. (2001). *Pattern Classification* (2nd ed.). Wiley-Interscience.
- [5] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning with Applications in R*. Springer.